

visualDITA_user_guide

2026-10-05



Contents

1	Visual DITA user guide.....	4
1.1	Contents.....	4
2	Checking your content.....	5
2.1	Where problems show.....	5
2.2	The grammar.....	5
2.3	The authoring rules.....	5
2.4	Your terminology.....	6
2.5	Your own rules (Schematron).....	6
2.5.1	Your rules files and their phases.....	7
2.5.2	Fixes.....	7
2.6	The whole project.....	8
2.7	When you edit the XML.....	8
2.8	In the Explorer, and anywhere.....	8
3	Your document types.....	10
3.1	Standard DITA: built in, and free.....	10
3.1.1	Which DITA version.....	10
3.1.2	Loading and unloading versions.....	10
3.1.3	A free renewal every year.....	11
3.2	Specializations: licensed.....	11
3.3	The DITA Document Types view.....	12
3.4	When a document type is not licensed.....	12
3.4.1	A topic of a type of your own.....	12
3.5	Your framework's templates and styling.....	13
4	Getting started.....	14
4.1	Install.....	14
4.2	Open your content.....	14
4.3	Find your way around.....	14
4.4	Create a topic.....	14
4.5	Before you start writing.....	15

5	Your glossary.....	16
5.1	What your glossary is.....	16
5.2	Glossary terms on the page.....	16
5.3	Adding a term.....	16
5.4	Consistent terminology.....	17
6	Maps.....	18
6.1	Building the map.....	18
6.2	Writing it as an outline.....	18
6.3	Arranging it.....	18
6.4	An entry.....	19
6.5	The map's metadata.....	19
6.6	Relationship tables.....	19
6.7	Conditions and branches.....	19
6.8	The DITA Map view.....	19
7	Publishing.....	21
7.1	Before your first publication.....	21
7.2	Publish a map or a topic.....	21
7.2.1	Your own DITA-OT setup.....	22
7.2.2	Settings.....	22
7.2.3	Your PDF.....	23
7.2.4	Publishing on a server.....	23
7.2.5	DITA-OT on your machine.....	23
7.3	Publishing another way.....	23
7.3.1	Write the stylesheet.....	24
7.3.2	Give it to DITA-OT.....	24
8	Review and compare.....	25
8.1	Review in the file.....	25
8.1.1	Comments.....	26
8.1.2	Tracked changes.....	26
8.1.3	No markup.....	27
8.1.4	Colour highlights.....	28
8.1.5	Maps.....	28
8.1.6	In the XML.....	28
8.1.7	Teams that review through Git only.....	28
8.2	Compare with Git.....	28
8.2.1	Single page.....	29

8.2.2	Side by side.....	29
8.2.3	Diffs from Source Control.....	29
8.2.4	What a comparison marks.....	30
8.2.5	Tracked changes and comments in a comparison.....	30
8.3	Not yet.....	31
9	Searching your content.....	32
9.1	Searching.....	32
9.2	Keeping to part of the project.....	32
9.3	What is searched.....	32
10	Tables.....	33
10.1	Insert a table.....	33
10.2	The table bar.....	33
10.3	Moving around and typing.....	33
10.4	Widths and alignment.....	33
11	Writing on the page.....	34
11.1	Typing, Enter and getting out.....	34
11.2	Styles: what the caret is in.....	34
11.3	The mini toolbar.....	34
11.4	Inserting elements.....	35
11.5	Extracting to a new topic.....	35
11.6	Pasting from Word and web pages.....	35
11.7	Converting a Word document.....	36
11.8	Numbered chapters and clauses.....	38
11.9	Links, images and reuse.....	38
11.10	Equations and drawings.....	39
11.11	Conditions.....	39
11.12	Attributes and the topic's metadata.....	39
11.13	Seeing the structure.....	40
11.14	Keyboard shortcuts.....	40

1 Visual DITA user guide

Visual DITA opens your DITA content in VS Code as a page you write on, the way you write in Word, while the files stay DITA: valid, lossless, yours.

Standard DITA is free. Every feature — the page editor, maps, tables, review, comparisons with Git, the checks and the agent tools — works on the standard OASIS DITA document types, on any number of computers, for any use, commercial authoring included.

Your own document types — specializations and constraints — are supported too: they come from PleoSoft as a licensed Visual DITA package, and that licence is the paid part. See [Your document types](#).

1.1 Contents

- 1 [Getting started](#) — install, open a folder, find your way around the window, create a topic.
- 2 [Writing on the page](#) — typing, Enter, the mini toolbar, inserting elements, links, images, reuse, renaming and moving files, conditions, the topic's metadata, keyboard shortcuts.
- 3 [Tables](#) — CALS and simple tables, the table bar, cells, rows, columns, widths.
- 4 [Maps](#) — the map as an outline, keys, relationship tables, the DITA Map view.
- 5 [Review and compare](#) — comments and tracked changes in the file, and comparing versions with Git on one page or side by side.
- 6 [Checking your content](#) — validation as you type, authoring rules, the Problems panel, the Project Health Report, Check References.
- 7 [Searching your content](#) — DITA Search: the text of your topics and maps by its words, kept to a kind of element or a map, each place a click from its paragraph.
- 8 [Your glossary](#) — glossary terms on the page, inserting them by their words, adding new ones, consistent terminology.
- 9 [Your document types](#) — standard DITA and its yearly renewal, licensed specializations, what "not licensed" means, your framework's templates and styling.
- 10 [Publishing](#) — a map published as HTML5 with the page's look, or as PDF, by the DITA-OT on your machine; the same look for DITA-OT run another way.

The [README](#) has the settings, the agent tools for AI assistants, and the requirements.

Questions and bug reports: <https://github.com/PleoSoft/visualdita-docs/issues>.

2 Checking your content

[User guide](#) › Checking your content

Visual DITA checks each topic while you write, against its document type (the grammar: which elements go where, which attributes and values are allowed), a set of authoring rules (what a good topic has, beyond what the grammar requires), and your project's own rules when it has them in Schematron.

2.1 Where problems show

- **On the page**, on the element they belong to: errors underlined in red; with the formatting marks (¶) on, a marker in the margin for each problem, warnings and notes included.
- **In the page's status line**: how many problems the topic has. Click it to go to the next one.
- **In VS Code's Problems panel (Ctrl+Shift+M)**: every problem with its line, for the topics you have open — as a page or as XML.

2.2 The grammar

The check follows the topic's own document type — standard DITA, or your licensed package — exactly: elements, their order, attributes and their values. A topic whose document type Visual DITA does not have is not checked against a stand-in, because every element of your own would be reported (see [Your document types](#)). `visualDita.validation` switches the grammar check off.

2.3 The authoring rules

Beyond the grammar, the rules point out:

Rule	Points out
<code>shortdesc</code>	a topic without a short description, or with one over 50 words. On the page, a faint Short description line under the title adds one (click it, or press Tab in the title); in the XML, the quick fix Add a short description does
<code>empty</code>	an empty title, paragraph, list item, note or section
<code>keys</code>	a key no map defines
<code>files</code>	a link or reference to a file that is not there
<code>reuse</code>	a conref or conkeyref that pulls an element of another type (a paragraph reusing a step: an element reuses its own type or a specialization of it), or content with elements your document type does not have

Rule	Points out
subject-scheme	a profiling value your subject scheme does not allow
alt	an image without alternative text
ids	an id used twice in a topic
external-scope	a link to a web address without <code>scope="external"</code>
draft	draft comments and required-cleanup, which the output leaves out
table	a table row with the wrong number of cells
tracked	tracked changes left in a file when the project does not track changes
terms	a word your glossary says not to use, with the term to use instead (see Consistent terminology)

Switch any of them off for a project with `visualDita.rulesOff`, for example `"visualDita.rulesOff": ["shortdesc", "draft"]`.

2.4 Your terminology

A project with a DITA glossary is checked for words its glossary entries say not to use, with the term to use instead and two fixes: see [Consistent terminology](#).

2.5 Your own rules (Schematron)

A project with its own checks written in Schematron (an Oxygen framework's, or your team's `.sch` files) gets them checked as you write, beside the grammar and the authoring rules. A topic or map is checked against:

- the rules its own `<?xml-model?>` names, as Oxygen writes it: `<?xml-model href="rules/house.sch" type="application/xml" schematypens="http://purl.oclc.org/dsdl/schematron"?>`;
- the rules `visualDita.schematron` lists for the project, relative to the workspace folder: `"visualDita.schematron": ["rules/house.sch"]`.

Their findings show where the others do: on the page, in the XML and in the Problems panel, as **Visual DITA Schematron** with the rules file's name and the pattern's id. On the page, a finding's message ends with the rules file's name, so you know the rule is your team's, not Visual DITA's. A rule's **severity** (ISO Schematron 2025), or else its **role**, is the finding's severity: **error** or **fatal** an error, **warning** a warning, **info** a note; an assert with neither is an error. Rules are ISO Schematron, of any edition; Schematron 1.5 rules, from before ISO, are said as such in the Visual DITA output, to be moved to ISO Schematron's namespace. Rules test DITA by its classes, as usual (`contains(@class, 'topic/p')`): the classes come from your document type, as in Oxygen. Rules written for XSLT 2 or 3 (`queryBinding="xslt2"` or `"xslt3"`) work as they are: their own XSLT functions and keys, the XSLT

libraries and Schematron files they include, abstract patterns, and the files they open (the topics a link names, read from where the document is).

A rules file is prepared the first time a document uses it, and again when it or a file it includes changes: a second or so, said in the status line. After that a document is checked in milliseconds, while you type. Rules that cannot be prepared are said once, the reason in the Visual DITA output.

Visual DITA: Check the Project Against Its Schematron Rules (Command Palette) checks every topic and map of the project its rules apply to, open or not, and says how many findings each rules file made; **Show Problems** lists them, file by file, each a click away. A file closed afterwards keeps its findings, as it is on disk. Their fixes are offered once the file is open.

2.5.1 Your rules files and their phases

The **DITA Document Types** view lists your rules files under **Schematron rules**: each one with the phase it checks with and what uses it (`visualDita.schematron`, or how many documents name it). Click one to open it; hover it for its phases. The checklist button on **Schematron rules** checks the whole project.

A rules file with phases (`sch:phase`) checks with its default phase (`defaultPhase`), or with every pattern when it names none. **Choose Phase...** (the filter button on a rules file, or its right-click menu) picks another: one of its phases, each with its patterns, or every pattern. The choice is kept for the project in `visualDita.schematronPhases`, the rules file relative to the workspace folder (`#ALL` for every pattern): `"visualDita.schematronPhases": { "rules/house.sch": "full" }`. Every check uses it: as you write and the whole project's. A phase the rules file does not have is said in the Visual DITA output, and that file checks nothing until you choose one it has.

2.5.2 Fixes

Where your rules give a finding fixes (Schematron QuickFix, `sqf:fix`), Visual DITA offers them:

- on a topic's page: right-click the marked element, or click the problem's marker in the margin; the fixes are at the top of the menu, under what the finding says;
- in the XML: as quick fixes on the finding (the light bulb, or Ctrl+.), the rules' default fix marked preferred.

A fix that asks for something (a user entry) ends with an ellipsis: choose it and Visual DITA asks, the rules' default filled in, before making it. A fix changes only what it names: an attribute added after the others, the words a fix replaces where they are written, the content it copies kept exactly as the file has it, its comments and entities too. Undo takes it back. With Track Changes on, a fix is your tracked change, to accept or reject as any other. Fixes follow the Schematron QuickFix specification: adding, deleting and replacing elements, attributes and text, replacing words in a text, generic fixes (one for each value), groups, the schema's own fixes, fixes calling others, and parameters of an abstract pattern the fix declares.

2.6 The whole project

Visual DITA: Project Health Report (Command Palette, or the DITA Map view's ... menu) looks at the whole doc set: each map's size in topics and words, links to files that are not there, keys no map defines, topics no map publishes, and images nobody uses; topics in DITA `.xml` files count too. Every file is a link that opens it. It is there at once, from what DITA Search already knows of your project, and counts what you have written but not yet saved.

Check References, the checklist button of the **DITA References** view, lists at the top of the view what does not resolve in the topic or map you are on: a file or image that is not there, an id a link or conref names that is not in its file, a key no map defines, a conref that names no element, and a link to a topic no map publishes (it goes nowhere once published). Each opens the source at its line; the count shows on the view. **Check References in the Whole Project**, in the view's menu, does the same for every topic and map, at once: both come from the reading of your project that DITA Search keeps, with what you have written but not yet saved. Links to web addresses, and those marked `scope="peer"` (another doc set), are left alone. A file renamed, moved or deleted outside VS Code (by Git, or another program) shows at once: the DITA Map view marks its entry as not there, the topics linking to it show the problem, and a check listed in the view runs again.

Visual DITA: Check Grammars in Workspace asks one more question of the whole doc set: which document types its files declare, and whether each one resolves to standard DITA or to a licensed package. Run it when a doc set arrives, before anyone concludes their own elements are not supported.

2.7 When you edit the XML

With a topic open as XML (**Open Source**), the same checks run, and Visual DITA completes the elements allowed where you type, their attributes and their values. Ctrl+click on a `keyref`, `conref` or `href` goes to its target, and hovering one shows what it resolves to.

- **F2** on an id, a key, or a reference to one renames it and every reference in the project; **Ctrl+Enter** in the rename box shows every change first.
- **Shift+F12** lists every place an id or a key is used, the key/element and `#!/element` forms included.
- The **Outline**, the breadcrumbs and sticky scroll follow the file's structure: topics, sections, figures and tables by their titles; a map's entries, headings and key definitions.

2.8 In the Explorer, and anywhere

The Explorer marks each topic and map: how many of its references do not resolve (what **Check References** lists), ! for a document type no Visual DITA package licenses, and a topic no map uses greyed. Hover one to read why. They come from what DITA Search knows of your project ([Searching](#)

[your content](#)), so they follow what you write, before you save, and files changed outside VS Code; `visualDita.explorerBadges` turns the marks off.

Go to Symbol in Workspace (Ctrl+T) finds a topic, section, figure or table by its title, a map by its title, a key, or an id (type `#` and the id) anywhere in the project.

Next: [Your document types](#).

3 Your document types

User guide › Your document types

Every DITA file names its document type in its DOCTYPE (`<!DOCTYPE task PUBLIC "-//OASIS//DTD DITA Task//EN" ...>`), or, written for RELAX NG, in an `<?xml-model?>` at its top, and Visual DITA works from that document type: it decides what the page offers, what the menus contain and what the check reports.

3.1 Standard DITA: built in, and free

The OASIS document types of DITA 1.0, 1.1, 1.2 and 1.3 — topic, concept, task, reference, glossary, map, bookmap, learning and training, machinery task, troubleshooting and the rest — are built in, and so is the DITA 2.0 draft. They cost nothing: every feature of Visual DITA works on them, on any number of computers, for any use, commercial authoring included. Nothing to install, nothing to configure.

3.1.1 Which DITA version

A DOCTYPE that names a version gets that version: `-//OASIS//DTD DITA 1.2 Task//EN` opens on DITA 1.2, whatever the rest of the project uses. Most DOCTYPEs name none (`-//OASIS//DTD DITA Task//EN`), and those open on your project's DITA version: DITA 1.3, unless the setting **DITA version** (`visualDita.ditaVersion`) says otherwise. Set it in the project's settings, `.vscode/settings.json`, so that everyone who works on the project gets the same:

```
{ "visualDita.ditaVersion": "1.2" }
```

The version decides what the page, the insert menus and the check offer: in a DITA 1.2 project you are not offered `<div>` or anything else DITA 1.3 added, and the check holds your documents to DITA 1.2. A document type your version does not have — a troubleshooting topic in a DITA 1.2 project — opens on the next version up that has it. In a workspace with several folders, each folder can have its own version.

The status line at the bottom of the page names the version a document got (`task · DITA 1.2`); click it to see why.

3.1.2 Loading and unloading versions

Each version is loaded in your project or not, under **Standard DITA** in the **DITA Document Types** view: DITA 1.0 to 1.3 are loaded unless you unload one, and the DITA 2.0 draft is not loaded unless you load it. A file of a version that is not loaded still opens, on base DITA, and says so: the status line reads **NOT LOADED** (`task · DITA 1.2 NOT LOADED: opened as base DITA`), the banner above the page has a **Load** button, and the check is off for that file. Nothing in the file is changed.

The choice is saved in the project's settings (`visualDita.standardDita` in `.vscode/settings.json`), so everyone who works on the project gets the same, and **New Topic from Template** offers only templates of versions that are loaded. In the same view, **Use as the Project's DITA Version** sets the version for files whose DOCTYPE names none.

DITA 2.0 is a draft, not yet an OASIS Standard. Visual DITA includes its current release (OASIS beta 03) for projects that want to start with it: load **DITA 2.0 draft**, and files whose DOCTYPE or schema names DITA 2.0 (`../OASIS/DT DITA 2.0 Concept/EN`, or `urn:pubid:oasis:names:tc:dita:rng:concept.rng:2.0`) open on it, with the page, the menus and the check following DITA 2.0. Its grammars may still change before OASIS approves it.

3.1.3 A free renewal every year

The standard DITA in Visual DITA works until 31 December: hover **Standard DITA** in the **DITA Document Types** view to see the date. Each year's renewal is free and comes from October:

- installed from the Marketplace or Open VSX, the update that carries it arrives by itself;
- otherwise, download it from visualdita.com — five files, one per DITA version, into one folder — and pick any one of them with **Import Visual DITA Package...** in the DITA Document Types view: the others come with it. It renews standard DITA for you, in every project; nothing is added to the project.

From 1 November Visual DITA reminds you, and the view shows the date. If the date passes without a renewal, a DITA file opens on a message instead of the page, with the download and **Open Source**. Your files are unchanged: editing them as XML, and using them with any other tool, is never restricted.

3.2 Specializations: licensed

Many organizations have document types of their own: specializations, which add elements (a hazard statement, a product-specific block), and constraints, which leave some out. Visual DITA supports them fully — the page draws your elements, the insert menus and the styles list offer them where your grammar allows, and the check follows your grammar — through a **Visual DITA package**.

A package is one file (`.vdpkg`) that Pleosoft prepares from your document types, written as DTDs or as RELAX NG: their grammars, compiled, with a licence in your name, signed. Add it with **Import Visual DITA Package...** in the **DITA Document Types** view: it is checked and copied into the `.dita` folder of your project, so it travels with the project for everyone who works on it. Visual DITA never reads your DTDs, RELAX NG or catalogs themselves.

A topic finds its document type in your packages by what it names: its DOCTYPE's public id (or, with none, the DTD's file name), or its `<?xml-model?>`: the URN your catalog gives the shell, or a path, which goes to the package whose shell sits at the end of that path. An OASIS identifier (`urn:oasis:names:tc:dita:rng:concept.rng`) opens on standard DITA, unless your catalog maps that very identifier to a shell of yours. When two packages have the same public id or URN, the first is used, and the DITA Document Types view says which.

This is the paid part of Visual DITA. To license your document types, contact Pleosoft at visualdita.com.

3.3 The DITA Document Types view

The **DITA Document Types** view (Visual DITA side bar) says what the editor uses: every document type your files declare, how many files use each and where it comes from — standard DITA and its version, your licensed package (and until when), or *not licensed / not loaded / unknown document type*, listed first — and below, what is loaded: your packages, and each version of standard DITA.

A licence that ends within a month is flagged there. A package can be unloaded for you, in this workspace (the file stays and can be loaded again), and **Visual DITA: Reload Grammars** reads the packages again without reopening anything.

3.4 When a document type is not licensed

A file whose document type is neither standard DITA nor in a licensed package still opens, and nothing in it is lost:

- it opens on the standard document type it is built on, so the standard content — titles, paragraphs, lists, tables — is on the page as usual;
- your own elements are kept exactly as they are in the file, and show as tags; they are not offered in the menus;
- the banner and the status line say **NOT LICENSED**, and the DITA Document Types view says why (no package has it, the licence ended, the package was unloaded...);
- the grammar check is off for that file, since it would report every element of your own.

Editing the file as XML, and using it with any other tool, is never restricted. **FALLBACK GRAMMAR** is the other readout you may see: the file names no document type Visual DITA knows (no DOCTYPE or `<?xml-model?>`, or a standard one it does not have), so it opens on the standard type matching its root element, or read-only as below when its root element is not a standard one.

Visual DITA: Check Grammars in Workspace lists, for the whole doc set, which document types resolve and which do not, with the files they concern.

3.4.1 A topic of a type of your own

A topic whose root element is yours — a `<warranty>` rather than a `<concept>` — has no standard type to open on. It is shown read-only instead, so the whole topic can still be read:

- your elements are labelled boxes around their content, and your inline elements are labelled where they are in the line;
- the DITA inside them — titles, paragraphs, lists, sections — is on the page as usual;
- the status line names each element the cursor is in, yours included;
- nothing can be changed on the page, and the ribbon is hidden; the banner says so, and **Open Source** opens the XML, which stays yours to edit.

Once the package that has your document type is licensed, the topic opens on its own grammar and can be edited.

3.5 Your framework's templates and styling

An Oxygen framework or a DITA-OT plugin usually also has topic templates and Author CSS. **Visual DITA: Use Framework Templates and CSS...** (also right-click a folder in the Explorer › **Visual DITA**) takes both from the folder you pick and adds them to the project's settings (**visualDita.templates**, **visualDita.css**): your templates are offered by **New Topic from Template...**, and your stylesheets style the page where they are plain CSS.

From an Oxygen framework, the stylesheets taken are the ones its **.framework** or **.exf** file uses: its default style and those always on, not its alternate or print styles, nor what it takes from Oxygen's own DITA framework. Visual DITA styles standard DITA itself, so Oxygen's own frameworks give their templates but not their CSS: your stylesheets are for what is yours, your own elements and your company's look.

The DITA Document Types view shows what is in use under **Templates and styling**: the template folders and the stylesheets, each a click away, and a setting that names nothing. Its buttons turn the styling off and on, and run **Use Framework Templates and CSS...** again. **Visual DITA: Turn Framework Styling On or Off** takes the styling away if a page ever looks wrong. In a folder VS Code does not trust (Restricted Mode), Author CSS from the workspace is not applied until you trust the folder.

Back to the [user guide](#).

4 Getting started

[User guide](#) › Getting started

4.1 Install

Install **Visual DITA** from the VS Code Marketplace (Extensions view, search for *Visual DITA*). It needs VS Code 1.106 or newer and nothing else: no Java, no XML extension, no server. The standard OASIS DITA document types are built in, so a doc set that uses them works at once, for free. Only publishing (HTML5, PDF) needs DITA-OT on your computer, installed once: see [Publishing](#).

4.2 Open your content

Open the folder that holds your DITA files (**File** › **Open Folder...**). Then:

- a topic (`.dita`) opens as a page you write on;
- a map (`.ditamap`, `.bookmap`) opens as an outline;
- a condition set (`.ditaval`) opens as a form.

The XML is always one click away: **Open Source** (the `</>` button at the top right of the page, or right-click › **Open source**) shows the file as XML, and **Open in Visual DITA** goes back. Edits made in either one appear in the other.

4.3 Find your way around

- 1 **DITA Map view** — the map that publishes the topic you are on, as a tree of titles. Click a title to open the topic. Keys defined in the map are listed at the top.
- 2 **DITA References** — what this topic uses (images, links, reused content) and what uses it; **Check References** lists what does not resolve.
- 3 **The ribbon** — as in Word: History, Styles, Font, Paragraph, Insert, Review, View and, where the file is in Git, Compare. Buttons that do not apply where the caret is are greyed out.
- 4 **The page** — your topic as it reads: titles, paragraphs, lists, notes, tables, figures and images. Type on it.
- 5 **The breadcrumb and the status line** — the elements the caret is in (click one to select that element), and on the right the file, its document type, and how many problems it has.
- 6 **Properties, Review and Topic** — the attributes of the element at the caret, the comments and tracked changes, and the topic's metadata. They follow whichever topic you are working on.

The side panes live in VS Code's secondary side bar by default. Set `visualDita.panes` to `page` to have them inside the page instead. **Ctrl+Alt+B** shows or hides the secondary side bar.

4.4 Create a topic

Right-click a folder in the Explorer › **Visual DITA** › **New Topic from Template...**, or run the command from the Command Palette (**Ctrl+Shift+P**).

The list has the standard DITA types, your project's templates (a `templates` folder, or the folders in `visualDita.templates`), and your personal ones. Give the topic a title: the file is named after it (*Cleaning the housing* becomes `cleaning-the-housing.dita`), its id and author are filled in, and it opens on the page. **New Map from Template...** does the same for maps and bookmaps, and **Save as Template...** turns the topic or map you have open into a template — for the project, or only for you.

4.5 Before you start writing

Set your name, for your comments and tracked changes: **File › Preferences › Settings**, search for `visualDita.author`. Left empty, it is your operating-system user name.

Save with **Ctrl+S** as usual. Visual DITA rewrites only the elements you changed: everything else in the file — indentation, comments, processing instructions, entities, attribute order — stays byte for byte as it was, so a diff of your commit shows your edit and nothing else.

Next: [Writing on the page](#).

5 Your glossary

[User guide](#) › Your glossary

Visual DITA works with the glossary your DITA project already has: its terms read on the page as the glossary writes them, a glossary term says what it means when you point at it, you insert terms by their words, add new ones from words you select, and every topic is checked for words the glossary says not to use.

5.1 What your glossary is

Your glossary is the glossary entries (`<glossentry>`) your maps give keys to, as `<term keyref>` and `<abbreviated-form keyref>` need them: a topic uses the glossary of the map it belongs to. An entry can be a file of its own or one of several in a `<glossgroup>`, its key then naming it (`(href="brakes.dita#abs")`); keys can be given by `<glossref>` (in a glossary map of its own, as Oxygen's guide recommends) or by `<keydef>`, in any map the root map reaches. Specialized glossary entries work as the standard ones do.

5.2 Glossary terms on the page

- A `<term keyref>` with no text of its own reads as the glossary's term, and an `<abbreviated-form keyref>` as the entry's abbreviation or acronym.
- **Point at a glossary term** to see what it means: its term and the glossary's definition.
- **Insert › Glossary term...** (right-click) lists the terms of your glossary by their words, each with its definition, and an acronym or abbreviation as its abbreviated form. The one you pick goes in by key; with words selected, they become the term's own text.

5.3 Adding a term

Select the words, right-click, **Add to Glossary...** Visual DITA asks the term (your words, to write as the glossary should) and what it means, then:

- writes the new entry where your glossary's entries are, as they are written (their `DOCTYPE`, their line ends), named after its key (`docking-station.dita`);
- gives it its key in the map that gives your glossary's entries theirs: one line after the last of them, written as they are (a `<glossref>` or a `<keydef>`, their other attributes such as `print="yes"` too). Nothing else of the map changes;
- makes your words a glossary term by that key, and offers to open the new entry.

A project with no glossary yet gets one laid out as Oxygen's guide recommends: a `glossary` folder beside the root map, a glossary map there giving the entry its key, and a map reference to it at the end of the root map. Every change to a map is checked against its document type first; where it would not be valid (a bookmap keeps its glossary in its back matter), Visual DITA says so and writes nothing: give it a glossary map, and entries are added there.

5.4 Consistent terminology

A glossary entry can say which forms of its term not to use, and why:

```
<glossentry id="usb-drive">
  <glossterm>USB flash drive</glossterm>
  <glossBody>
    <glossAlt>
      <glossSynonym>stick</glossSynonym>
      <glossStatus value="prohibited"/>
      <glossUsage>This is too colloquial.</glossUsage>
    </glossAlt>
  </glossBody>
</glossentry>
```

A topic that says "stick" is told so: *"stick": use "USB flash drive", as the glossary says (prohibited). This is too colloquial.* Two fixes are offered, on the page (right-click the paragraph, or click the problem's marker in the margin) and as quick fixes in the XML:

- **Use "USB flash drive"** writes the term in place of the word, capitalised where the word was.
- **Link the glossary term** puts a `<term keyref>` to the glossary entry in its place, where the document type allows one there. It reads as the glossary's term.

With Track Changes on, a fix is your tracked change.

- **Words are matched whole**, in any case, but a form in capitals (an acronym) only in capitals. Code, preformatted text and draft comments are left alone, and so is a form inside one you may use ("flash" in "USB flash drive").
- **What "do not use" means** is `visualDita.terminology.avoid`: by default `prohibited`, `obsolete` and `deprecated`. DITA leaves the values to your project.
- **The check is the `terms` authoring rule**: `visualDita.rulesOff` switches it off, as any other (see [Checking your content](#)). The agent tools' check reports these words too.
- DITA 2.0 has no `glossStatus`, so its glossaries mark nothing to avoid.

6 Maps

[User guide](#) › Maps

A map (`.ditamap`, or a `.bookmap`) opens as an outline of titles: what the publication contains, in its order. Titles come from the topics themselves and from keys, so the outline reads like a table of contents.

6.1 Building the map

The ribbon's **Add** group adds, after the selected entry:

- **Topic...** — a reference to a topic that exists, picked from the project; pick several and they are added in name order;
- **New topic...** — a new topic from a template, created and referenced in one go, in the folder of the topics around it;
- **Heading** — a heading that groups entries without a topic of its own (`topichead`);
- **Key** — a key definition, for text or a target you reuse by key;
- **Map...** — another map, referenced from this one (or several);
- **More ?** — the other entries your map's document type allows there: a group, and your own entry types (a term map's term groups, say). One that references a file asks for it; a group that must hold an entry asks for its first.

Right-click › **Add** offers the same.

6.2 Writing it as an outline

A map can be written before its topics exist, as a table of contents is typed:

- **Enter** in the map's title starts it with a heading, its title ready to type;
- **Enter** at the end of a heading's title adds the next heading, as the next line (under it, when its entries are shown); **Tab** and **Shift+Tab** nest it or bring it out, while you keep typing; **Escape** leaves the title.

Then give the headings their topics. **Create topic...**, on a heading or right-click, makes its topic from a template, titled as the heading, in the folder of the topics around it; the heading becomes the reference to it. Right-click › **Create topics for the ... entries without a file...** makes them all at once, from one template.

6.3 Arranging it

Drag an entry to move it, with everything under it. The **Arrange** group moves the selected entry up or down, indents it under the entry above or outdents it, and removes it. **Outline** folds and unfolds the whole tree.

6.4 An entry

Select an entry to work on it. **Open** opens its topic beside the map; **Rename** changes its navigation title; **Change file...** points it at another file; **Create topic...**, on an entry with no file yet, makes its topic. The bar at the bottom of the page sets its keys and key scope, and the Properties pane shows all its attributes.

Inline into parent topic... (right-click an entry under another topic's entry) is Extract's reverse: the entry's topic moves into its parent's topic, and its file goes. It asks how when both can be:

- **as a section**, when the topic is that simple: no sections or topics of its own, no prolog, and used nowhere but under its parent. Its title becomes the section's, its short description a paragraph; a numbered entry makes a numbered section, so its number stays;
- **as a nested topic**, whole: its id, short description and prolog kept, at the end of the parent topic.

Its entry leaves the map, its own entries taking its place; every link and reuse that pointed at it, in maps and topics, points where it went. What the parent's document type does not take (a task nested in a concept) is not offered.

6.5 The map's metadata

The map's own metadata — its `<topicmeta>`, a bookmap's `<bookmeta>` — is in the **Topic** pane, as **Map info** (**Book info** for a bookmap): the map's short description, created and revised dates, authors, keywords, the product and its version, and for a book its rights owner, copyright years, ISBN and publisher. It describes the publication as a whole, and publishing tools pass some of it on to the topics. Each field is added where the map's document type allows it, and is not offered where it does not. Like a topic's metadata it is not on the page: **Show on the page as elements**, in the pane, puts it back as it is in the file.

6.6 Relationship tables

Rel. table ? adds a relationship table and edits it in place: rows of topics that link to each other when the map is published.

6.7 Conditions and branches

Conditions ? previews the map through a condition set. An entry that publishes a branch through its own condition set (`ditavalref`) shows it on its row, and a topic under such a branch previews through that condition set when you open it.

6.8 The DITA Map view

The **DITA Map** view in the Visual DITA side bar shows the map that publishes the topic you are working on, and follows you as you open other topics; click a title to open that topic. With several root maps it lists them all. Opening a map shows that map alone, and so does **Choose Map...** in its

title bar, or right-clicking a map in the Explorer › **Visual DITA** › **Show in Map Panel**; **Show All Maps**, beside them in the title bar (or the first entry of **Choose Map...**), lists them all again. The map shown alone is also what **DITA Search** keeps to (see [Searching your content](#)). **Filter** (the funnel in its title bar) finds a map or a topic by its title as you type, the others hidden; **Ctrl+Alt+F** in the view opens VS Code's own find there (it marks the matches, unless VS Code is set to filter); everywhere else it opens DITA Search. Its ... menu has **Edit Map**, the **Project Health Report** (see [Checking your content](#)) and **Refresh**.

Maps carry comments and tracked changes too: see [Review and compare](#).

Next: [Review and compare](#).

7 Publishing

[User guide](#) › Publishing

Visual DITA publishes your maps with DITA-OT, DITA's open-source publishing engine, installed on your machine: as HTML5 or as PDF, both looking the way your topics look on the page.

7.1 Before your first publication

DITA-OT is free, and it is not part of Visual DITA: it is installed once, on your computer.

- 1 Download DITA-OT, version 4 or later, from dita-ot.org/download, and unzip it anywhere you like, your Documents folder for instance. It unzips into a folder named `dita-ot-` and its version.
- 2 DITA-OT runs on Java, version 17 or later. If your computer has no Java yet, install it, for instance from adoptium.net.
- 3 Publish (below). The first time, Visual DITA says it needs DITA-OT: click **Choose DITA-OT Folder...** and pick the folder you unzipped. Visual DITA remembers it, for every project. (**Get DITA-OT**, beside it, opens the download page.)

If Java is missing, Visual DITA says so when you publish. The details, for whoever installs DITA-OT another way, are at the end of this page ([DITA-OT on your machine](#)).

7.2 Publish a map or a topic

Publish... publishes the file it is on: right-click a map or a topic in the Explorer and choose **Visual DITA › Publish...**, or click the rocket at the top of the editor where the map or topic is open. In the **DITA Map** view, the rocket on a map's or a topic's row (when you point at it, or its right-click menu) publishes that map, or that topic alone with the keys of the map it is listed in; when the view shows one map, the rocket in its title bar publishes it. **Visual DITA: Publish...** in the Command Palette publishes the map that view stands for. On a topic, **Publish Map...** (the same menus) publishes the map it belongs to, asking which when it is in several.

Then choose:

- the output: **HTML5** (web pages) or **PDF**, both with Visual DITA's look, or any other output your DITA-OT has (its plugins' among them), as DITA-OT makes it; for your first PDF, the paper (A4 or Letter), asked once;
- the conditions, when your project has DITAVAL files: one of them, or none (everything in the map).

DITA-OT publishes your files as they are saved. When topics, maps or DITAVAL files you have open have unsaved changes, Visual DITA asks first: **Save and Publish**, or **Publish Saved Versions** (your changes left out, still unsaved).

Visual DITA remembers both for each map. The result goes to `out/<map>/html5` or `out/<map>/pdf` in your project (`html5-print` for HTML5 published with `print.ditaval`), and **Open** opens it. Each

publication empties its folder first. HTML5 is laid out as help pages are: every topic's page has the map's contents in a panel beside it, the topic shown marked (DITA-OT's `nav-toc`, laid out by Visual DITA's stylesheet). A map whose topics are outside its folder (`maps/manual.ditamap` with its topics in `topics/`) publishes every page into the output, as deep as the map is among them, so that every link works (DITA-OT's `generate.copy.outer`): its start page is then `maps/index.html`, and **Open** opens it there. A root map in the topmost folder of its topics has its start page at the top.

What DITA-OT says goes to the Problems panel and onto the page, by the element it is about (on the map when DITA-OT names no place), as **Visual DITA publishing (DITA-OT)** with DITA-OT's message number. Publishing again replaces them. DITA-OT's whole output is in the **Visual DITA: DITA-OT** output.

A topic is published alone, into `out/<topic>/...`: with the keys of the map it belongs to resolved in it (Visual DITA gives DITA-OT a small map that holds the topic and refers to that map for its keys, removed afterwards), and nothing else of that map. HTML5 has the pages it links to beside it, so its links work. A topic in no map is published without keys. The DITA-OT parameters for a topic published alone are a setting of their own, `visualDita.publishing.topic`.

7.2.1 Your own DITA-OT setup

A project already set up for DITA-OT is published as it is set up. **Publish...** first offers the deliverables of your DITA-OT project files (XML or JSON, anywhere in the project, their includes followed), each by its name, its output type and its maps, and publishes the one you choose exactly as its file says (`dita --project=<file> --deliverable=<id>`): nothing of Visual DITA's added, its output in `out/` as DITA-OT run in the project folder puts it. A deliverable without an id cannot be published alone: its whole file is. A map's own **Publish...** (right-click it in the DITA Map view) offers the deliverables that publish that map. Below them, **A map, as HTML5, PDF or another output...** is Visual DITA's own way, above.

7.2.2 Settings

What Visual DITA publishes with are DITA-OT's own parameters, and they are yours to set, in your project's settings:

- `visualDita.publishing.html5` and `visualDita.publishing.pdf`: DITA-OT's parameters for each output, name → value (DITA-OT's documentation lists them: <https://www.dita-ot.org/dev/parameters/>). Visual DITA's defaults for HTML5 are `"args.copycss": "yes"` and `"args.csspath": "css"` (the stylesheet copied beside the pages), `"nav-toc": "full"` (the contents beside every topic) and `"generate.copy.outer": "3"` (every page in the output); for PDF, none. Yours go with Visual DITA's: give one another value to change it, or `null` to leave it out.
- `visualDita.publishing.look`: Visual DITA's look (on), or DITA-OT's own (off). On, Visual DITA adds the one parameter only it can, where its look is: the stylesheet (`args.css`) for HTML5, the theme (`theme`) for PDF.

Kept in the project, Visual DITA's publications in `publishing/project.xml` follow these settings: when they change, the next publication rewrites them, so a server publishes as you do. Publications of your own in that file stay as you wrote them.

7.2.3 Your PDF

The PDF looks like the page: its fonts, the headings' blue, notes with their coloured bar, code, tables, the task's sections named as the page names them, the page numbers at the foot and the map's title at the head. Two things stay DITA-OT's: a table's outer frame, and a note's label at the start of its text rather than above it. Your topics' own colours and shading (a Word document's, kept as `outputclass`) reach HTML5 only.

Kept in the project (below), the PDF is made with `publishing/theme.yaml`, a DITA-OT theme of your own: it starts with your paper and extends `publishing/visualdita-theme.yaml`, Visual DITA's look, which Visual DITA writes again when the look changes. What you set in yours — the paper, margins, a logo on the cover, the header and footer, any style — goes over Visual DITA's and stays yours; its comments show where to start, and DITA-OT's documentation has every key (<https://www.dita-ot.org/dev/topics/pdf-themes.html>).

7.2.4 Publishing on a server

The first time you publish in a project, Visual DITA asks whether to keep what you publish in the project. Kept, each map you publish, with its output and its conditions, is a deliverable in `publishing/project.xml`, DITA-OT's own project file, and Visual DITA's look is beside it: the stylesheet for HTML5 (`publishing/visualdita.css`) and the themes for PDF (`publishing/theme.yaml` over `publishing/visualdita-theme.yaml`). Visual DITA then publishes through that file, so a DITA-OT anywhere — a build server, a colleague's machine — publishes your maps exactly as you do:

```
dita --project=publishing/project.xml
```

Each deliverable's output goes to its own folder in DITA-OT's output folder (`out/` unless the server gives another with `-o`). Commit the `publishing` folder with your topics; at the same moment Visual DITA offers to keep `out/` out of Git. The file is yours too: Visual DITA only adds a deliverable when you publish a new combination, and leaves the others, yours included, as they are. A project whose `publishing/project.xml` is already there (a colleague's) is published through it without asking. **Copy Server Command** in the message after publishing copies the command.

7.2.5 DITA-OT on your machine

Publishing needs DITA-OT 4 or later, and Java 17 or later to run it. Visual DITA uses the DITA-OT in `visualDita.ditaOt` (its folder, the one holding `bin`), else the one in `DITA_HOME`, else the `dita` on your PATH. When it finds none, or no Java to run it, it says so before asking anything: **Get DITA-OT** opens DITA-OT's download page, **Choose DITA-OT Folder...** sets `visualDita.ditaOt`.

7.3 Publishing another way

DITA-OT run any other way — from the command line, from a build, or from another VS Code extension such as DitaCraft — can give its HTML5 output the same look.

7.3.1 Write the stylesheet

Run **Visual DITA: Write Publishing Stylesheet...** from the Command Palette and choose where to save it (`visualdita.css` in your project, by default). It holds:

- the look of your content as the page shows it: titles, paragraphs, notes and their colours, lists, steps and the names of a task's parts ("Before you begin", "Procedure"...), code, tables, figures, links;
- the colours, shading, capitals and alignment your topics keep as `outputclass` (a converted Word document's);
- the numbers of numbered chapters and clauses, as the page counts them: each topic's from your maps, its numbered sections and legal lists after it (see [Numbered chapters and clauses](#)).

The numbers and colours are your project's at the moment you write it: write it again when your maps or colours change.

7.3.2 Give it to DITA-OT

DITA-OT's HTML5 output uses it given three arguments; the message after writing it copies them for you:

```
dita -i your.ditamap -f html5 --args.css=path/to/visualdita.css --args.copycss=yes --args.csspath=css
```

With **DitaCraft** installed, **Use in DitaCraft** in that message adds them to this workspace's `ditacraft.ditaOtArgs`, replacing a stylesheet given there before; its HTML5 publishing then uses Visual DITA's look.

DITA-OT writes some labels itself ("Note:", "Figure 1."): the stylesheet styles those rather than adding its own, so they read as DITA-OT words them. Its PDF output does not use CSS: give it the theme instead, `--theme=publishing/theme.yaml`, once **Publish...** has kept a PDF in the project.

8 Review and compare

[User guide](#) › Review and compare

Visual DITA offers two ways to see what changed in a topic. They suit different teams, and they work together.

- **Review in the file.** Comments and tracked changes are stored in the topic itself, in the markup Oxygen XML uses. They travel with the file, need no Git, and a reviewer in Oxygen sees the same comments and changes.
- **Compare with Git.** Nothing is added to the file. Visual DITA compares the versions Git already keeps — the last commit, any earlier one, another branch, what you staged — and shows the differences on the page, on a single page or side by side.

Your team...	Use
sends topics out for review, works with Oxygen users, or wants comments on the text	review in the file
keeps its content in Git and reviews commits or pull requests	Compare, with Track changes switched off for the project
does both	both: a comparison reads a file's tracked changes as accepted, so the two never mix on the page

- [Review in the file](#)
 - [Comments](#) · [Tracked changes](#) · [No markup](#) · [Colour highlights](#) · [Maps](#) · [In the XML](#) · [Teams that review through Git only](#)
- [Compare with Git](#)
 - [Single page](#) · [Side by side](#) · [Diffs from Source Control](#) · [What a comparison marks](#) · [Tracked changes and comments in a comparison](#)
- [Not yet](#)

8.1 Review in the file

Comments, tracked changes and colour highlights are processing instructions — `<?oxy_comment_start ...?>`, `<?oxy_insert_start ...?>`, `<?oxy_delete ...?>`, `<?oxy_attributes ...?>` — exactly as Oxygen XML writes them. A topic reviewed in Visual DITA opens in Oxygen with its comments and changes intact, and the other way round. Markup you do not touch is written back byte for byte.

Publishing tools ignore this markup. Comments and highlights never reach the output; inserted text is ordinary content and deleted text is kept inside its marker, so a topic published with changes still pending reads as if they were accepted. Accept or reject them before you publish.

Your name on comments and changes is `visualDita.author`; left empty, it is your operating-system user name.

The Review pane has two tabs, **Comments** and **Changes**. By default it sits in the Visual DITA side bar (the secondary side bar) and follows the topic you are working on; with `visualDita.panes` set to `page` it opens inside the page instead.

8.1.1 Comments

Add one on the selected words, or at the caret when nothing is selected:

- **Comment** in the ribbon (Review group) or in the mini toolbar over a selection,
- **Ctrl+Alt+M** (Cmd+Alt+M on a Mac),
- right-click → **Add comment**,
- **+ Comment** in the Review pane.

Type the comment and press **Comment** (or Ctrl+Enter). To comment on a whole element — a note, a table, a section — select it by clicking its name in the breadcrumb at the bottom of the page, then add the comment.

On the page a commented passage is highlighted in yellow, and the right margin has an icon for each thread, with the number of comments still open in it. Click the icon, the highlight's mark in the overview ruler on the right, or the comment in the Review pane, to go to it.

In the Review pane each comment shows its author and date, with:

- **Reply** — replies thread under the comment, and replies to replies nest.
- **Resolve / Reopen** — resolving the first comment of a thread resolves its replies too, and reopening it reopens them; a reply can be resolved on its own. A resolved thread leaves the page (no highlight, icon or ruler mark) and stays in the pane. **Show resolved** brings resolved threads back onto the page, in grey.
- **Delete** — removes the comment and its replies; the commented text stays.
- **To draft** — turns the thread into a DITA `<draft-comment>` in the topic, replies included (each with its author and date), for teams that keep review notes as DITA. **To comment** turns such a draft-comment back into a comment thread. Draft-comments are listed in the pane too, and an authoring rule notes each one: DITA-OT leaves them out of the output unless the build keeps drafts.

Reply, Resolve, Reopen, Delete and the conversions are also on the right-click menu of commented text.

8.1.2 Tracked changes

Track changes (ribbon, Review group; the right-click menu; the Changes tab of the Review pane) switches recording on and off. Oxygen keeps that choice in the file — `<?oxy_options track_changes="on"?>` — and so does Visual DITA: a file saved with tracking on opens with tracking on, in either editor, and switching it changes the file.

What is recorded, each change with your name and the time:

- typing, deleting, and typing over a selection (shown as one **Replaced** change),
- pasting, inserting and deleting elements, splitting a paragraph with Enter, table edits,

- attribute changes made in the Properties pane or from the right-click menu, keeping the original value,
- formatting — making words bold, putting them in a `<uicontrol>` — recorded as the old words replaced by the formatted ones,
- moving content by drag and drop, recorded as deleted in one place and inserted in the other.

Deleting inside your own insertion simply removes the text; deleting anywhere else, including inside another author's insertion, is recorded as a deletion. Colour highlights and comments are never recorded, and neither are undo and redo.

A deletion across paragraphs (or list items, cells, other elements) is recorded in each of them — the words deleted in the first and the last, whole paragraphs in between — and the paragraphs are not joined, not even once the deletion is accepted: the review markup has no way to record a join. For the same reason, Backspace at the start of a paragraph (and Delete at its end) moves the caret to the neighbouring paragraph instead of joining the two. Deleting commented words keeps the comment, around the deleted words.

On the page inserted text is green and underlined, deleted text red and struck through, where it was. An attribute change has no colour on the page; hovering the element shows the old and new values. Every block with a change gets a thin grey bar in the left margin, and the overview ruler marks each change. Hovering a change says who made it.

In the Review pane (Changes tab) each change reads *Inserted*, *Deleted*, *Replaced* or *Attributes of ... changed*, by whom and when, with the text or the old and new values. From there, or by right-clicking the change on the page:

- **Accept / Reject** one change. A replacement is accepted or rejected as one; rejecting a deletion brings the deleted content back, formatting included; rejecting an attribute change restores the old values.
- **Note** — a note on the change ("why"), as Oxygen has; **Edit note** changes it, and emptying it removes it.
- **Accept all / Reject all** — every change in the topic, by every author, at once.

8.1.3 No markup

No markup (ribbon, Review group) shows the topic as if every tracked change were accepted and without comments: deletions hidden, insertions as plain text, no highlights, margin icons, change bars or ruler marks. The file keeps everything, you can go on editing, and the Review pane still lists it all. The button is there only when the topic has comments or tracked changes. It is not remembered: reopening the topic, or anything that reloads the page (a change to the file from outside, reloading grammars), shows the markup again.

8.1.4 Colour highlights

The colour palette in the ribbon's Font group and in the mini toolbar paints a background colour on words — eight colours, and **No colour** to remove it. It is Oxygen's highlight markup: a reviewer's marker pen, shown on the page, never published and never tracked. **Clear formatting** removes it too.

8.1.5 Maps

A map carries comments and tracked changes the same way. In the map page, **Comment** (ribbon) or right-click → **Comment on entry** comments an entry; with **Track changes** on, added, removed and moved entries and attribute edits are recorded. Entries show a coloured bar — green inserted, red deleted (a struck ghost row), purple attributes, yellow commented — and the right-click menu has Reply, Resolve, Remove, Accept and Reject. No markup, highlights, the margin and the ruler are topic-only.

8.1.6 In the XML

Opened as XML (right-click → **Open Source**), a topic or map shows its review markup in the gutter and the overview ruler — comments yellow, insertions green, deletions red, attribute changes purple — with the details on hover.

8.1.7 Teams that review through Git only

A team that reviews through Git does not want review markup in its files. Switch `visualDita.trackChanges` off in the project's settings (`.vscode/settings.json`):

```
{ "visualDita.trackChanges": false }
```

Track changes then disappears from the ribbon, the right-click menu and the Review pane, and nothing can switch it on — not even a file saved with Oxygen's tracking option, which is left as it is in the file. Tracked changes already in a file still show, to accept or reject, and the authoring rule `tracked` warns about each file that holds some, in the page and in the Problems panel. Comments stay available: they are not changes to the text.

8.2 Compare with Git

Comparing needs VS Code's built-in Git (on by default) and a topic in a Git repository. The toolbar's last group, **Compare**, then has two buttons, **Single page** and **Side by side**. Both ask which version to compare with:

- **Last committed version** — the last commit that changed the file,
- an earlier commit from the file's history (message, date, author),
- **Branches** — the file as it is on another local or remote branch.

Your side of a comparison is the topic as it is in the editor, unsaved changes included.

8.2.1 Single page

Single page shows the changes on your page: new words in green, removed ones struck through in red where they were; a whole new or removed paragraph gets a dashed outline and a + or – in the right margin, so it never looks like a note. The bar at the top names the version and counts the changes. The page is read-only while it compares, so the ribbon steps aside; **Close comparison** (or Single page again) gives it back. **Side by side** in the bar opens the same two versions next to each other.

8.2.2 Side by side

Side by side opens the version you picked and your file in a diff tab — `task.dita (1a2b3c4) ↔ task.dita` — both shown as pages:

- left, **Committed version**: what is gone or changed since, struck through in red;
- right, **Your version**: what is new, in green.

The two sides scroll together, kept level on the paragraphs they share, however much longer one of them is. Each side's overview ruler marks its changes; click one to go there, and the other side follows. Both sides are for reading — no ribbon, menus or side panes. Tracked changes are shown as accepted there, comments and colour highlights as you have them. On the right:

- **Edit** opens your file in its own tab, in the same editor group; the diff stays one tab over and follows what you change, before you save.
- **Single page** opens your file with the same changes on one page.

8.2.3 Diffs from Source Control

Clicking a changed file in the Source Control view opens VS Code's text diff of its XML. To read it as pages, pick **Visual DITA Editor** in the editor list at the top right of the diff, or run **View: Reopen Editor With...** from the Command Palette. To have every `.dita` diff open that way, run **View: Reopen Editor With...** once and choose **Configure default editor (diff only) for '*.dita'...** → Visual DITA Editor.

Every diff Git offers works:

Diff	Left	Right
a changed file (Changes)	the committed version	your file
a staged file (Staged Changes)	the last commit	what you staged
a commit in the Graph or the Timeline	the commit before	that commit

When neither side is your working file — staged changes, a commit against its parent — the bars say **Earlier version** and **Later version**, **Edit** opens the file itself, and there is no Single page. Which side is the earlier one comes from Git, not from the order the pages opened in.

8.2.4 What a comparison marks

On the page	Means
green, underlined	words that are new
a dashed green outline, + in the right margin	a paragraph, list item or other element that is new
red, struck through	words that went, drawn where they were
struck through in a dashed red outline, – in the right margin	a whole paragraph or element that went, drawn where it stood
a green row, a struck red row	a table row added, a table row removed (on a single page, drawn among the rows)
dotted violet underline	the same words formatted otherwise: bold, italic, a phrase element such as <code><uicontrol></code> , a link or key pointing elsewhere
dashed violet outline	an element whose attributes changed — a note's type, an <code>@outputclass</code> , a condition — or that became another element with the same words (a paragraph into a note, or wrapped in one); hover to see which
violet frame around an image or formula	an image, formula or drawing (MathML, SVG) that is different in the other version: another picture, a changed formula
green frame; a red one on the earlier version	an image, formula or drawing only the later version has; only the earlier one had
<i>Image pic.png</i> , struck through	an image (or formula, drawing) that went, named where it stood

Paragraphs, list items and table cells are matched by the words they share, so an edited paragraph reads word by word, and a paragraph removed beside an edited one does not make both look rewritten. Tables are compared as tables: rows by what the whole row says, cells in the same column; a new column shows as new cells. The overview ruler marks new in green, removed in red and formatting, attribute, element, image and formula changes in violet, and the summary counts them all ("2 images changed, 1 element turned into another").

Not marked:

- the topic's metadata (`<prolog>`) — it is not on the page; the Topic pane shows it,
- a move, which reads as removed in one place and new in the other,
- comments (they are review markup, not content) and XML comments,
- a table's `@cols`, which only follows its columns.

8.2.5 Tracked changes and comments in a comparison

A comparison reads every version as if its tracked changes were accepted: inserted text counts as there, deleted text as gone, on both sides. Accepting or rejecting a change therefore shows in a

comparison only through the text it changes, and a tracked change and a comparison are never drawn in the same colours at once:

- While a comparison is open, **single page or side by side** (and diffs from Source Control), the tracked changes are shown as accepted — the bar says how many — and closing it brings them back.
- **Comments and colour highlights stay** as you had them, on both sides of a diff too; with No markup on, they are hidden there as well.

8.3 Not yet

- A comment's text cannot be edited once sent: delete it and write it again.
- There is no next/previous change command and no per-author filter; Accept all and Reject all cover every author.
- Track changes has no keyboard shortcut; Ctrl+Alt+M (comment) is the only review shortcut.
- In a map: no No markup, highlights, margin or ruler.

9 Searching your content

[User guide](#) › Searching your content

DITA Search, at the top of the Visual DITA side bar (**Ctrl+Alt+F**, or **Visual DITA: Search DITA...**), searches the text of your topics and maps as a reader reads it: never inside a tag or an attribute, and with every place found said for what it is, a title, a step, a note, a table cell.

9.1 Searching

Type in the box: what is found shows as you type, grouped by topic (its title and its file), each place with the words around it and the words found marked. Click a place, or press Enter for the first: the topic opens with the caret in that paragraph, title or step.

- **Every word** you type must be there, in any order, with or without capitals and accents. A word also finds the longer words it begins: **valve** finds *valves*, **instal** finds *install* and *installation*.
- **"A phrase in quotes"** is found exactly as written.
- **Titles first:** a title that matches comes before a short description, which comes before body text.
- **A word misspelt:** when nothing is found, the nearest word your project uses is searched instead (**presure**: *pressure*), and the line above the results says so.

9.2 Keeping to part of the project

- **A kind of element:** titles, short descriptions, notes, steps, lists, tables, code, the glossary. Your own specialized elements count as what they specialize, and a paragraph in a note is a note's.
- **A map:** when the **DITA Map** view shows one map alone (you opened it, or chose it there), DITA Search keeps to that map's topics, and says so under the box; **Show all maps** searches the whole project again. With every map listed in the DITA Map view, the whole project is searched, topics no map uses yet included ([Maps: the DITA Map view](#)).

9.3 What is searched

Every topic and map in your workspace folders, except your template folders. Very large files (over 5 MB) are not searched.

You never have to wait for it or refresh it: what you type is found before you save, and files that change outside VS Code are found too, a Git pull or branch switch, a folder renamed or deleted, even while VS Code was closed. Nothing is added to your project. If what is found ever looks out of date, **Visual DITA: Rebuild Project Index** (also in the DITA Search view's ... menu) reads your whole project again.

The same knowledge of your project makes **Where Is This File Used**, the **DITA References** view, **Go to Symbol in Workspace** (**Ctrl+T**), the Explorer's marks, the **Project Health Report** and **Check References** answer at once.

10 Tables

[User guide](#) › Tables

DITA has two kinds of table, and Visual DITA draws both as real tables.

- A **table** (CALS) can merge cells across rows and columns, and has a title and column specifications.
- A **simple table** (`simpletable`), and the choice tables and properties tables built on it) is a plain grid.

10.1 Insert a table

The **table** button in the Insert group inserts a 3 × 3 CALS table with a header row wherever the document type allows one; the arrow beside it offers a simple table instead.

10.2 The table bar

With the caret in a table, a bar above it names the kind of table and offers what you can do to it:

- **Insert** — a row above or below, a column left or right.
- **Merge** — the cell with the one to its right or the one below, or split a merged cell again (CALS tables only; greyed out in a simple table, which cannot merge).
- **Delete** — the row, the column, or the whole table.
- **Header row** — make the row a header row, or a body row again.

10.3 Moving around and typing

- **Tab** goes to the next cell, **Shift+Tab** to the previous one. Tab in the last cell adds a row.
- **Enter** is a new line inside the cell, in any cell.
- **Ctrl+Enter** leaves the table and continues in a paragraph after it.
- Typing wraps the text in the cell and the row grows, as in Word; the columns keep their widths.

10.4 Widths and alignment

Drag a column border to set its width. It is written the way DITA expresses it: `colspec/@colwidth` in a CALS table, `@relcolwidth` in a simple table. Alignment (`@align`, `@valign`), frames and rules (`@frame`, `@colsep`, `@rowsep`) and key columns (`@keycol`) are drawn on the page, and set in the Properties pane.

Next: [Maps](#).

11 Writing on the page

[User guide](#) › Writing on the page

Visual DITA knows your document type: wherever the caret is, it offers only the elements allowed there, and the keys do what that place expects. You write; the structure stays valid.

11.1 Typing, Enter and getting out

- **Enter** continues what you are writing: a new paragraph, a new step, a new list item, the next definition. In an empty list item or step it steps out of the list, the way a second Enter does in Word. In preformatted text (code, screens, `<lines>`) and in a table cell, Enter is a new line. Where no paragraph may follow, Enter makes what your document type allows next, such as the next author in the prolog. Where it allows several things, a small menu at the caret offers them, the same kind first: Enter again takes it, Escape makes nothing.
- **Related links**: a link that shows its target's title has nothing in it to type into, so a click selects it whole and its link bar opens; typing gives it text of its own. **Enter** on a related link (selected, or with the caret in its own text) works as in a list: a new link appears right after it, in the same list, and Visual DITA asks what it points to. Cancel, and it stays as *Link without a target*: give it one later with **Change...**, or press **Enter** on it to leave the related links (it goes, and you are back at the end of the body). **Insert** → **Related link...** (right-click) adds a related link to the topic, the first one too.
- **Ctrl+Enter** leaves the element you are in — a note, a section, a table, a code block — and continues in a paragraph after it.
- **Backspace** and **Delete** in an empty element remove the element whole, instead of leaving an empty shell.
- **Tab** and **Shift+Tab** indent and outdent list items, and move between table cells. In a topic's title, Tab goes on to its short description.
- A topic without a short description shows a faint **Short description** line under its title: click it (or press Tab in the title) and type. Left empty, it goes again, and the file is not changed.

11.2 Styles: what the caret is in

The **Styles** box on the ribbon names the element at the caret (*Paragraph, Step, Title...*). Its menu turns that element into another one allowed there (a paragraph into a note, a paragraph into preformatted text), and inserts an element after it, at the end of its parent, or at the caret. The breadcrumb at the bottom of the page shows the whole chain of elements; click one to select it.

11.3 The mini toolbar

Select some words and a small toolbar appears over them: bold, italic, underline, code, a highlight colour, clear formatting, a link, a comment, and **Condition...** to tag the words for an audience or product.

Format painter (the brush in the Font group) copies the formatting at the caret — bold, the element such as `<uicontrol>`, the colour — onto the next words you select; double-click it to paint several times, **Esc** stops.

11.4 Inserting elements

The ribbon's **Insert** group has the common ones: note, section, step, code block, table, link, image, equation, reuse. Right-click › **Insert** has everything the document type allows at that spot, grouped by where it goes — inline at the caret, after the current element, at the end of its parent — with **All elements... (Styles list)** for the rest.

The same right-click menu turns the element into another, deletes it (also **Ctrl+Shift+Delete**), shows where the topic is used, makes the element reusable (gives it an id to conref), and opens the XML at that place. **Make ... reusable** moves the element into a topic you pick, and says so when that topic's document type does not allow it there.

11.5 Extracting to a new topic

Select paragraphs, lists, tables, or a whole section, and right-click › **Extract selection to new topic...**; or right-click a section › **Extract Section to new topic....** The new topic is written beside this one (a section's title is its title; a selection asks for one), and what points at the content follows it, the way a refactoring updates every call site:

- every map that uses this topic gets the new topic under it, after its other entries;
- links and reuse, in this topic and in the others, that pointed into what moved now point at the new topic; a link to the section itself opens the new topic;
- in the new topic, links to what stayed behind still point here.

A nested topic is extracted whole — its short description, its prolog, the topics nested in it — and keeps its id: links into it keep pointing where they did, in the new file. With a map to hold it, nothing is left in its place; in no map, a link to the new topic is. A reference by key to an element that moved (`key/element`) cannot follow: it is said, and listed in the Visual DITA output. The new topic is a plain topic, or one of your topic's own type when its content needs it (your own elements).

The reverse is on the map: right-click an entry › **Inline into parent topic...** (see [Maps](#)).

11.6 Pasting from Word and web pages

Copy from Word, a web page or Google Docs and paste on the page: what you get is DITA, not the source's formatting.

- Paragraphs, bold, italic, underline, links, lists (nested as they were) and tables (with their header row) come across as themselves.
- A paragraph that starts with **Note:**, **Tip:**, **Important:**, **Caution:** or **Warning:** becomes a note of that type.

- Headings become sections, each titled by its heading, where the paste lands in a body that takes sections (an empty paragraph in a concept's or topic's body); elsewhere they come in as bold paragraphs.
- A whole document pasted into a new, still empty topic gives the topic its title (Word's **Title** style, or the document's only first-level heading).
- Images in what you copied are saved in an **images** folder beside the topic. A caption (Word's **Caption** style) makes its image a figure titled by it, or gives the table above or below it its title; the "Figure 1:" in front is left to publishing, which numbers them.
- Footnotes become DITA footnotes where they are referenced, **Quote** paragraphs long quotes, and lines in a code font (Courier, Consolas) a code block.
- What Word shows but is not content stays out: its table of contents, the numbers of numbered headings and lists, and text deleted with Track Changes on. Text inserted with Track Changes on is kept, as if accepted.
- The look DITA keeps comes along, as a converted Word document keeps it: text colour and capitals (as **outputclass**), the highlighter (the page's own highlight), a centred or right-aligned paragraph. The black and grey a web page gives all its text is not a colour, and stays out.

11.7 Converting a Word document

Right-click a folder in the Explorer › **Visual DITA** › **Import Word Document...** and pick one or more Word documents (**.docx**): each goes into a new folder in the one you right-clicked, named after it. A Word document already in your project converts where it is: right-click it › **Visual DITA** › **Convert Word Document to DITA...** (several at once too), and the new folder is beside it.

Either way the folder holds:

- a map, titled by the document's **Title** (else its first lines when they are centred and bold, else the running title of its page footer or header), its topics nested as the headings are, and the copyright the footer states;
- a topic for each of those headings, named after it (**draining-the-pump.dita**); deeper headings become sections. A heading over one numbered list becomes a task: the list its steps, **Before you begin** its prerequisites, the text before the list its context and what follows its result — unless the list states rather than instructs: definitions ("Boat" means...) or clauses titled in bold (**Notice** -- ...) keep it a concept. The others are concepts;
- the images, in **images/**.

Where the topics start is yours to choose. For a document with more than one level of headings, Visual DITA asks, showing how many topics each choice gives: a topic for each Heading 1, for each Heading 1 and 2, and so on down to every heading, nested in the map as the headings are; the headings below the ones you pick become sections. It proposes your last choice, else the document's own: each Heading 1 when it has several, each Heading 2 as well when it has one or none (**Word Conversion: Split**, **visualDita.wordConversion.split**, sets what it proposes until you have chosen once). A choice that would keep Word's numbers as text says so: a numbered clause made a topic is counted apart from the clauses left in the text above it. Chapters now and finer later works too: a section becomes a topic of its own with **Extract selection to new topic**.

The content comes across as a paste from Word does (lists, tables, notes, figures, footnotes, links; Word's table of contents left out, its tracked changes accepted), and more of what Word says:

- **Numbers.** Numbered chapters and clauses become what DITA numbers, with no number typed in: a numbered heading a numbered entry of the map, a clause under it a numbered section (titled when the clause is a title, around its text when it is a sentence), a deeper clause an item of a legal list. See [Numbered chapters and clauses](#). When counting would not give the numbers Word shows (a numbering that starts past 1, a clause 1.1 with no clause 1), they are kept as text instead, as Word shows them, and that is said. **Word Conversion: Numbers** ([visualDita.wordConversion.numbers](#)) keeps them as text always, or leaves them out.
- **Lists** stay whole: a paragraph that goes on an item is in that item, a sublist nested in its item. DITA numbers every list from 1: a list Word starts at another number is said.
- **A footnote on a heading** goes at the start of the text under it: a DITA 1.3 title takes none.
- **Tables made with tabs.** Lines lined up with tabs become a table when everything says they are one: the same tab stops on every line, a bold header row or a column of codes and figures ("400", "3 days"), and not the letter or number of a list in front. A line broken by hand in a cell reads as one line. Columns Word laid side by side on a page are read one after the other, and the title and headings every page repeats are kept once.
- **Headings a document only shows.** A document's own styles are its headings where they say so: a style with an outline level, or a bold one used only for short lines (a form's own heading style, say). A heading on two lines is one; a heading typed in capitals is put in a sentence's case, the words the text itself writes in capitals kept. A document without heading styles has its short bold lines as headings, the bigger the higher.
- **Forms typed without Word's lists.** Numbers typed before a tab ("1.", "a.") become lists, nested by how far they are indented; a paragraph indented with an item goes on with it. The lines typed above the document's title (a form number, the issuer), a "Page 1 of 2", and at the end a copyright or a line repeating them are the page's header and footer typed in the text: left out, the copyright going to the map.
- **The look DITA keeps:** text colour, shading and capitals (as [outputclass](#), which the page shows and a publishing style can map), the highlighter (the page's own highlight), alignment, an image's size and placement, a table cell's shading. Fonts, sizes and page breaks are the publishing style's. Hidden text and a form field's placeholder text are left out.
- **Equations** become equations, as **Equation** inserts them: select one on the page to edit it.
- **Drawings that are not pictures** — shapes, charts, SmartArt — cannot come across as they are: each is marked where it was, by its name, in a red dashed box (DITA's [required-cleanup](#)). Save it as a picture in Word and put the picture there. A text box's words come across as text. Pictures in EMF or WMF (from Visio or older Word) are kept but do not show in a browser or most PDF output: the conversion says which; save them as PNG instead.

What was read from how the document looks (a table from tabs, headings from a style or from bold lines, lists from typed numbers, the header lines left out) is said in the Visual DITA output. Links to places in the document go to the topic that holds them. Nothing that exists is written over: a folder of that name already there makes it [Manual 2](#), and a file that appears in the new folder while it is being written stops the conversion. The map opens when it is done. What did not come across (text boxes, drawings, fields) is said when it is done, and listed in the Visual DITA output. It is a first draft of your document in DITA: read it through on the page, and move what belongs elsewhere.

11.8 Numbered chapters and clauses

DITA keeps no numbers in the text: publishing counts them. What is numbered is marked with `outputclass`:

- an entry of a map, `numbered`: numbered among the numbered entries around it, after the number of the numbered entry it is in (6, 6.1);
- a section of a topic, `numbered`: after the topic's number (6.1, 6.2);
- the items of a list, `legal`: after the number of the section, or topic, they are in (6.2.1); a legal list in an item one level deeper (6.2.1.1).

The page shows these numbers as they are counted: on the map, and in a topic before its title, its numbered sections and its clauses. They are not in the text, so they cannot be typed over or go stale. A section added with **Section** beside numbered sections is numbered too, and the ones after it renumber; a new item of a legal list is numbered by being there. A converted Word document is written this way. Your publishing style prints the numbers when it counts `numbered` and `legal` as described here.

11.9 Links, images and reuse

- **Link (Ctrl+K)** links the selected words to a topic, or to an element in it, picked from your project; keys work too. A link's text and target resolve on the page: a link with no text of its own shows the title of what it points at, by key too. With the caret on a link, a small bar shows where it goes, with **Open**, **Change...**, **Key...** and **Remove link**. **Open** on a link to an element opens its topic with the caret in that element.
- **Images: Image** on the ribbon picks a file, or paste or drop an image onto the page — it is saved in an `images` folder next to the topic. Drag a corner to resize it; the size is written as DITA expresses it (`@width`, `@scale`, `@scalefit`).
- **Videos and sounds** (`<object>`) show as a card saying what they are and where, with a player: a sound or video file of your project plays on the page (mp3, wav, mp4), and **Open** opens it beside the topic. A video another site plays (a YouTube embed, Oxygen's `outputclass="iframe"`) plays in the browser: **Open in browser**; YouTube refuses its player inside VS Code. Click the card (not the player) and a small bar edits it: its **Address**, **Replace...** to pick another video or sound of your project, what it plays (**Video**, **Sound**, **Embedded page**: the `outputclass` Oxygen writes), its **Width** and **Height**, and **Delete**. A double-click on the card puts you straight in the address. Only what you change is written.
- **Images, videos and sounds from a web server** wait for your word: loading one tells that server the topic was opened. A line above the page names each server, with **Load This Time** (this topic, until you close it), **Always from ...** and **Never**; until then each is a placeholder that says where it comes from. Your answers are kept in your own settings (`visualDita.remoteImagesAllowed`, `visualDita.remoteImagesBlocked`), never in a project's; `visualDita.remoteImages` decides for the servers you have not answered about: ask, show or never. A plain `http`: image, video or sound never loads. Anything else the page is not allowed to load is refused and noted in the status bar, the details in the Visual DITA output.
- **Reuse** inserts content written once elsewhere, by key (`conkeyref`) or from a file (`conref`). Reused content shows on the page as it will be published, read-only; one click opens its source. An element reuses its own type or a specialization of it (a note can reuse a hazard statement, not the other way round), so the element inserted is the one the reused content can fill; Visual DITA says so when the reused content has elements your document type does not.
- **Keys**: text defined by a key (`<keyword keyref="product"/>`) shows its value, resolved through the

map and its key scopes.

- **Glossary terms:** right-click › **Insert › Glossary term...** picks a term of your glossary by its words; point at a glossary term to see what it means; select words and **Add to Glossary...** to make them a new entry. See [Your glossary](#).
- **Renaming and moving files:** rename or move a topic, map, image or folder in the Explorer, and Visual DITA offers to update what points at it — the maps' entries and key definitions, links, reused content, images — and, for a file that changes folder, its own references. **Show Changes First** shows every file as it is and as it would be, before anything changes; **Always** and **Never** stop the question (the `visualDita.updateReferences` setting). References through keys need nothing: only the key's definition in the map changes. Only the references change: every other byte of each file stays as it was. A file open with unsaved changes gets the update in its editor, to save with the rest of your changes.
- **Renaming an id or a key:** right-click an element with an id › **Rename id...**, or an element with a `keyref` › **Rename key...** (in a map, right-click the entry that defines it): every reference in the project follows — links and reuse by path (`file.dita#topic/element`, `#!/element`), by key (`key/element`), and a key's every `keyref` and `conkeyref`, `scope.key` from outside its scope included. The same name in another key scope is another key, and is left alone. Edited as XML, **F2** does the same (see [Checking your content](#)). A new name already taken in the topic, or in the key's scope, is refused as you type it. A subject scheme's values are not renamed this way: the documents' attribute values would stay behind.

11.1 Equations and drawings

0

MathML formulas and SVG drawings render as themselves. **Equation** inserts one inline, on its own line, or as a numbered equation figure. Right-click a formula to write it in LaTeX, with a live preview (it is stored as MathML, with the LaTeX kept for the next edit), or to edit the MathML itself.

11.1 Conditions

1

Select words and choose **Condition...** in the mini toolbar to give them an audience, a platform or a product — or a condition of your own document type (an attribute domain: a `@jobrole` your specialization adds). The values offered are your subject scheme's, or the ones the project already uses. A value written the generalized way (`props="jobrole(admin)"`) counts as `jobrole="admin"`. **Conditions ?** on the ribbon previews the page through a condition set (`.ditaval`) — what it excludes is greyed out — and **Show condition tags** labels every conditional element with its values.

11.1 Attributes and the topic's metadata

2

The **Properties** pane shows the attributes of the element at the caret: allowed values as lists, profiling attributes with the values in use, everything else as a text field. The topic's metadata (its `<prolog>`) is not on the published page, so it is not on yours either: it is in the **Topic** pane — created

and revised dates, authors and keywords, the product and its version. **Show on the page as elements** puts it back on the page as it is in the file. A map's metadata is in the same pane, as Map info (see [Maps](#)).

11.1 Seeing the structure

3

- **Tags** shows every element's name on the page.
- ¶ (formatting marks) shows where each paragraph ends, which elements are empty, non-breaking spaces, and the authoring warnings in the margin.

11.1 Keyboard shortcuts

4

Keys	Does
Enter	a new paragraph, step or list item; a new line in preformatted text and table cells; elsewhere, what the document type allows next
Ctrl+Enter	leave the element (note, section, table...) and continue after it
Tab / Shift+Tab	indent or outdent a list item; next or previous table cell
Ctrl+B, Ctrl+I, Ctrl+U	bold, italic, underline
Ctrl+K	link the selection
Ctrl+Alt+M	comment on the selection
Ctrl+Shift+Delete	delete the element at the caret
Ctrl+Z, Ctrl+Y	undo, redo

On a Mac, use Cmd instead of Ctrl.

Next: [Tables](#).